Laboratório Funcional · Aula 12

João Marcos Moreira Nunes · RA 24055168-2 · Turma Esoft-6S-N-C
Gerado em 19/10/2026, 19:11:21 · tempo registrado no cronômetro: 0:00

Preparação: Programar com funções matemáticas

def area(r): return 3.14159 * r * r

Pura

def agora(): return datetime.now()

Impura

def total(lista): return sum(lista)

Pura

def registra(msg): print(msg)

Impura

def sorteia(): return random.randint(1, 6)

Impura

def poe(lista, x): lista.append(x); return lista

Impura

0.1 Escolha duas funções que você marcou como impuras e explique, para cada uma, o que a impede de ser pura.

agora() é impura porque consulta data e hora atuais, então a mesma chamada pode devolver resultados diferentes. registra(msg) também é impura porque produz um efeito externo ao imprimir na tela. sorteia() depende de aleatoriedade e poe() modifica a lista recebida.

Prática 1: Funções puras e formas funcionais

Previsão: O que as linhas 1, 4 e 7 vão imprimir?

Linha 1: A-1 e A-2. Linha 4: 14, pois g(4)=12 e f(12)=14. Linha 7: [7, 3, 9] [3, 7, 9], pois sorted cria uma lista nova ordenada.

Saídas (original e com compor(g, f))

```
Saída original:
1: A-1 A-2
2: True
3: 8
4: 14
5: [4, 9, 16]
6: 29
7: [7, 3, 9] [3, 7, 9]
8: [3, 7, 9]

Trocando compor(f, g) por compor(g, f), a linha 4 imprime 18.
```

1.1 Por que proximo_id("A") devolve valores diferentes para a mesma entrada? Por que isso dificulta os testes (p. 637)?

proximo_id("A") devolve valores diferentes porque altera o contador global a cada chamada: primeiro A-1, depois A-2. O resultado depende do estado anterior e da ordem das chamadas, o que dificulta testes repetíveis e isolados.

1.2 compor(f, g)(4) e compor(g, f)(4) deram valores diferentes. Calcule os dois à mão e explique a diferença.

compor(f,g)(4)=f(g(4)): g(4)=3×4=12 e f(12)=14. compor(g,f)(4)=g(f(4)): f(4)=6 e g(6)=18. A ordem importa porque a composição aplica primeiro a função mais interna.

1.3 Qual é a diferença entre sorted(notas) e notas.sort()? Qual dos dois segue o estilo funcional, e por quê?

sorted(notas) cria uma nova lista ordenada e mantém notas inalterada. notas.sort() altera a própria lista. sorted combina melhor com o estilo funcional porque evita modificar o dado original.

Prática 2: Scheme: tudo é lista

Previsão: (car '(A B C))

A

Previsão: (cdr '(A B C))

(B C)

Previsão: (cons '(A B) '(C D))

((A B) C D)

Saídas (original e sem o apóstrofo)

```
Saída esperada com o apóstrofo:
22
14
25
8
negativo
B
(B C)
(A B C)
((A B) C D)
B

Sem o apóstrofo, Scheme tenta avaliar A como operador/procedimento e ocorre erro porque A não está definida como função.
```

2.1 Para que serve o apóstrofo (')? O que o Scheme tentou fazer com (A B C) quando ele não estava lá?

O apóstrofo faz com que '(A B C) seja tratado como dado literal, sem avaliar a lista. Sem ele, Scheme tenta executar a lista como chamada de função, usando A como operador; como A não é um procedimento definido, ocorre erro.

2.2 Escreva uma expressão que devolva o terceiro elemento de '(A B C) usando só car e cdr. Teste no OneCompiler e cole aqui a expressão e o resultado.

```
Expressão: (car (cdr (cdr '(A B C))))
Resultado: C
```

Prática 3: Recursão e recursão em cauda

O que vi no passo a passo

n = 4: comum em 8 passos, até 3 multiplicações esperando (expressão de até 30 caracteres); em cauda em 6 passos, nada esperando (até 17 caracteres).

Previsão: O programa em Scheme chama conta-ate recursivamente um milhão de vezes. Vai funcionar? E a mesma função em Python?

No Scheme, a recursão em cauda deve funcionar para um milhão de passos quando o interpretador otimiza chamadas em cauda. Em Python, a chamada recursiva tende a ultrapassar o limite de recursão e gerar RecursionError, pois Python não faz essa otimização.

Saída do Scheme

```
Saída esperada no Scheme:
#t
#f
2432902008176640000
2432902008176640000
1000000
```

Saída do Python (as últimas linhas bastam)

```
Saída esperada/parcial em Python:
500
RecursionError: maximum recursion depth exceeded
A chamada com 1.000.000 excede o limite de recursão.
```

3.1 No passo a passo, o que cresce na recursão comum e não cresce na recursão em cauda? Por quê?

Na recursão comum, as chamadas anteriores ficam na pilha porque ainda há multiplicações a fazer na volta. Na recursão em cauda, a chamada recursiva é a última operação e o acumulado já vai como parâmetro; com otimização, a pilha não precisa crescer.

3.2 Por que o Scheme fez um milhão de chamadas e o Python parou com RecursionError? (Dica: releia o quadro sobre a otimização da chamada em cauda e a mensagem de erro do Python. O livro fala do Scheme na p. 656.)

Scheme otimiza chamadas em cauda e reutiliza o espaço da pilha, permitindo um milhão de chamadas. Python não faz essa otimização e tem limite de profundidade recursiva, então gera RecursionError.

Prática 4: Mão na massa: três funções sobre listas

Saída final (com os três ok)

```
Saída esperada após completar as funções:
tamanho: 3  ok
soma-quadrados: 14  ok
pares: (2 4 6)  ok
```

4.1 Cole aqui as suas três funções (tamanho, soma-quadrados e pares).

```
(define (tamanho lis)
  (if (null? lis) 0 (+ 1 (tamanho (cdr lis)))))

(define (soma-quadrados lis)
  (if (null? lis) 0 (+ (* (car lis) (car lis)) (soma-quadrados (cdr lis)))))

(define (pares lis)
  (cond ((null? lis) '())
        ((even? (car lis)) (cons (car lis) (pares (cdr lis))))
        (else (pares (cdr lis)))))
```

4.2 Qual foi a parte mais difícil? Onde você usou null?, car e cdr em cada função?

O principal é definir o caso-base da lista vazia. null? verifica se acabou; car acessa o primeiro elemento; cdr passa o restante à recursão. tamanho conta um por elemento, soma-quadrados soma o quadrado de car, e pares usa even? para decidir se inclui o elemento com cons ou pula para o restante.

Prática 5: Haskell: padrões, compreensões e preguiça

Previsão: squares é uma lista infinita. O que você acha que take 5 squares imprime? E primeiro 42 (div 1 0)?

take 5 squares imprime [0,1,4,9,16]. primeiro 42 (div 1 0) devolve 42, porque primeiro usa só o primeiro argumento e Haskell não avalia o segundo argumento desnecessário.

Saídas (original e com member 17)

```
Saída esperada (GHC):
2432902008176640000
[1,2,4,7,14]
[2,2,4,7,8,9]
[0,1,4,9,16]
[0,1,4,9,16,25,36,49]
True
42

Se descomentar member 17 squares, a busca não termina: 17 não é quadrado e a lista é infinita.
```

5.1 Como o Haskell consegue imprimir parte de uma lista infinita? E por que primeiro 42 (div 1 0) não deu erro de divisão por zero?

Haskell avalia a lista infinita sob demanda: take 5 solicita só cinco elementos. primeiro 42 (div 1 0) retorna 42 porque a função ignora o segundo argumento; pela avaliação preguiçosa, a divisão por zero nunca é calculada.

5.2 O que aconteceu com member 17 squares? Por que ele nunca termina, enquanto member 16 squares terminou? Como você corrigiria? (Dica: squares está em ordem crescente. Quando já dá para desistir da busca? O livro mostra uma solução na p. 670.)

member 16 squares termina ao encontrar 16 e retorna True. member 17 squares nunca termina porque 17 não aparece entre os quadrados. Como a lista é crescente, pode-se corrigir a busca para retornar False quando o elemento atual ultrapassar o valor procurado.

Prática 6: Você já usa funcional

Previsão: O que mais10(5) e [1, 2, 3].map(vezes(5)) vão imprimir?

mais10(5) imprime 15. [1, 2, 3].map(vezes(5)) imprime [5, 10, 15].

Saída do JavaScript

```
Saída esperada em Node.js:
15
[ 5, 10, 15 ]
56
{ nome: 'Ana', nota: 7 } { nome: 'Ana', nota: 8 }
```

Saída do Java

```
Saída esperada em Java:
14
18
1050.0
[CABO, MOUSE, TECLADO]
```

6.1 somador(10) já terminou de executar quando mais10(5) é chamado. Como a função devolvida ainda "lembra" o 10?

A função devolvida mantém um fechamento (closure) com o ambiente onde foi criada. Ele conserva n=10 mesmo depois que somador termina, então mais10(5) calcula 5+10=15.

6.2 Por que f.compose(g).apply(4) e f.andThen(g).apply(4) deram resultados diferentes? Qual deles corresponde ao compor(f, g) da Prática 1?

compose aplica primeiro g e depois f: g(4)=12, f(12)=14. andThen aplica primeiro f e depois g: f(4)=6, g(6)=18. f.compose(g) corresponde a compor(f,g), isto é, f(g(x)).

Prática 7: O laço vira pipeline

7.1 Primeiro exercício escolhido (1, 2 ou 3): cole o seu código reescrito.

```
Exercício 1 — Java com Stream:
import java.util.*;
public class Main {
  public static void main(String[] args) {
    List<Integer> nums = List.of(3, 8, 5, 12, 7, 6);
    int soma = nums.stream().filter(n → n % 2 == 0).mapToInt(n → n * n).sum();
    System.out.println(soma);
  }
}
Saída: 244.
```

7.2 Segundo exercício escolhido: cole o seu código reescrito.

```
Exercício 2 — JavaScript com filter e map:
const alunos = [{nome:"Ana",nota:8.5},{nome:"Bruno",nota:5.0},{nome:"Carla",nota:7.0},{nome:"Davi",nota:9.5}];
const aprovados = alunos.filter(aluno ⇒ aluno.nota ⩾ 6).map(aluno ⇒ aluno.nome.toUpperCase());
console.log(aprovados.join(", "));
Saída: ANA, CARLA, DAVI
```

Saídas das suas versões

```
Exercício 1 (Java): 244
Exercício 2 (JavaScript): ANA, CARLA, DAVI
```

7.3 Qual versão é mais fácil de ler: a com laço ou a funcional? E qual é mais fácil de depurar (por exemplo, para ver um valor intermediário)?

A versão funcional costuma ser mais curta e deixa clara a sequência de operações. O laço pode ser mais fácil para depurar passo a passo, pois permite inspecionar cada iteração e o acumulador. Depende da familiaridade com a linguagem.

Fechamento: O que fica do paradigma funcional

Uma função pura pode ler uma variável global que muda.

Falso

Em Scheme, a recursão em cauda não faz a pilha crescer.

Verdadeiro

O Python otimiza a recursão em cauda.

Falso

Em Haskell, [1 ..] só é calculada à medida que é usada.

Verdadeiro

filter, map e reduce dispensam laço e variável acumuladora.

Verdadeiro

Currying transforma uma função de vários parâmetros numa cadeia de funções de um parâmetro.

Verdadeiro

8.1 Qual versão é mais fácil de entender, e por quê? O livro estima que um programa funcional costuma levar cerca de 2 vezes mais tempo para executar (p. 678). Em que tipo de sistema você aceitaria esse custo, e em qual não?

A versão funcional em Haskell é mais concisa: cria a lista de 1 a n, aplica map para elevar ao cubo e sum para somar. A versão C é explícita e familiar. Aceitaria custo de desempenho em relatórios e ferramentas internas se clareza e manutenção forem prioridades; evitaria em sistemas de tempo real ou processamento crítico se medições mostrarem impacto relevante.

8.2 Em uma frase: o que mais surpreendeu você nesta atividade?

O que mais surpreendeu foi perceber que uma lista infinita pode ser usada calculando somente a parte necessária, como acontece em Haskell.

8.3 Qual é o erro de impressão, e como corrigir? O que a mensagem do compilador diz sobre o tipo que ele inferiu?

O erro é o filtro b <- h: deveria ser b <= h. Correção: sort (h:t) = sort [b | b <- t, b <= h] ++ [h] ++ sort [b | b <- t, b > h]. O compilador reclama porque b <- h tenta percorrer h como lista, mas h é um único elemento; há incompatibilidade de tipos.